



kubernetes

KUBERNETES PROJECT

• • • • •
• • • • •
• • • • •
• • • • •

Contributors:

김동진 이영훈 장규혁 정성현

CONTENTS

01 ETCD Backup & Upgrade

02 역할 기반 제어

03 노드 관리

04 파드 생성

05 Kubernetes Controller & Networking

06 NetworkingPolicy & Ingress

07 Resource 관리

ETCD Backup & Upgrade

[ETCD Backup]

https://127.0.0.1:2379에서 실행 중인 etcd의 **snapshot**을 생성하고 snapshot을 **etcd-snapshot.db**에 저장합니다.

스냅샷을 복원합니다. etcdctl을 사용하여 서버에 연결하기 위해 다음 TLS 인증서/키가 제공됩니다.

CA certificate: /etc/kubernetes/pki/etcd/ca.crt

Client certificate: /etc/kubernetes/pki/etcd/server.crt

Client key: /etc/kubernetes/pki/etcd/server.key

[ETCD Backup](#)

※ **Volume snapshot** 참고

0. 사전 작업

```
# change root
sudo -i

# install etcd-client
apt install etcd-client
```

1. snapshot 생성

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save etcd-snapshot.db
```

```
Snapshot saved at etcd-snapshot.db
root@master:~# ls
etcd-snapshot.db
```

2. snapshot 복구

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot restore etcd-snapshot.db
```

```
root@master:~# ls
default.etcd  etcd-snapshot.db
```

```
root@master:~# tree default.etcd/member
default.etcd/member
├── snap
│   └── 000000000000000001-0000000000000001.snap
├── db
└── wal
    └── 0000000000000000-0000000000000000.wal
```

ETCD Backup & Upgrade

[Cluster Upgrade]

마스터 노드의 모든 Kubernetes control plane 및 node 구성 요소를 버전 **1.33.11-1.1** 버전으로 업그레이드합니다.

master 노드를 업그레이드하기 전에 **drain** 하고 업그레이드 후에 **uncordon**해야 합니다.

"주의사항" 반드시 Master Node에서 **root권한**을 가지고 작업을 실행해야 한다.

[Cluster Update](#)

※ 해당 버전이 이미 1.33.11-1.1이므로 본 프로젝트에서는 Cluster Version을 1.34.7-1.1로 Upgrade 하였음.

0. 현재 버전 확인

```
cka@master:~/k8s$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	control-plane	12d	v1.33.11
worker-1	Ready	<none>	12d	v1.33.11
worker-2	Ready	<none>	12d	v1.33.11

1. 업그레이드 버전 확인

```
# update api list
sudo apt update

# Check available kubeadm versions
sudo apt-cache madison kubeadm
```

```
kubeadm | 1.34.7-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.6-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.5-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.4-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
kubeadm | 1.34.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.34/deb Packages
```

2. kubeadm Upgrade

```
# Package Update
sudo apt-mark unhold kubeadm && \
sudo apt-get update && sudo apt-get install -y '1.34.7-1.1' && \
sudo apt-mark hold kubeadm

# Check installed kubeadm version
kubeadm version

# Verify the upgrade plan
sudo kubeadm upgrade plan
```

ETCD Backup & Upgrade

```
Upgrade to the latest stable version:

COMPONENT           NODE           CURRENT        TARGET
kube-apiserver       master         v1.33.11       v1.34.7
kube-controller-manager master         v1.33.11       v1.34.7
kube-scheduler       master         v1.33.11       v1.34.7
kube-proxy           1.33.11       v1.34.7
CoreDNS              v1.12.0       v1.12.1
etcd                 master         3.5.24-0       3.6.5-0
```

2. kubeadm Upgrade

```
# kubeadm Upgrade
sudo kubeadm upgrade apply v1.34.7-1.1
```

3. Drain the Node

```
# Node Drain
kubectl drain master --ignore-daemonsets
```

4. Upgrade kubelet and kubectl & Restart

```
# Upgrade
sudo apt-mark unhold kubelet kubectl && \
sudo apt-get update && sudo apt-get install -y kubelet='1.34.7-1.1' kubectl='1.34.7-1.1'
sudo apt-mark hold kubelet kubectl

# Restart
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

5. Uncordon the node

```
kubectl uncordon master
```

6. Upgrade 확인

```
cka@master:~/k8s$ k get node
NAME          STATUS    ROLES          AGE   VERSION
master        Ready     control-plane  12d   v1.34.7
worker-1      Ready     <none>         12d   v1.33.11
worker-2      Ready     <none>         12d   v1.33.11
```

※ 참고

- 1. 쿠버네티스 정책상 워커 노드의 kubelet은 마스터보다 최대 2단계 낮아도 호환됩니다.
- 2. 하지만 안정성과 신규 기능 활용을 위해 마스터와 버전을 동일하게 맞추는 것을 권장합니다.
- 3. 워커 노드들도 동일한 방식(unhold -> install -> hold)으로 순차적 업데이트를 진행하십시오.

역할 기반 제어

[ServiceAccount, Role, RoleBinding 생성]

애플리케이션 운영중 특정 namespace의 Pod들을 모니터할수 있는 서비스가 요청되었습니다.
api-access 네임스페이스의 모든 pod를 view할 수 있도록 다음의 작업을 진행하시오.

- 1. api-access라는 새로운 **namespace**에 pod-viewer라는 이름의 **Service Account**를 만듭니다.
- 2. podreader-role이라는 이름의 **Role**, podreader-rolebinding이라는 이름의 **RoleBinding**을 만듭니다.
- 3. 앞서 생성한 ServiceAccount를 API resource **Pod**에 대하여 **watch, list, get**을 허용하도록

1. NameSpace 생성

```
# Create namespace
kubectl create namespace api-access

# Verify namespace creation
kubectl get namespace
```

```
cka@master:~/k8s$ kubectl get namespace
NAME          STATUS    AGE
api-access    Active    9s
```

2. ServiceAccount 생성

```
# Create ServiceAccount
kubectl create serviceaccount pod-viewer -n api-access

# Verify ServiceAccount creation
kubectl get serviceaccount --namespace api-access
```

```
cka@master:~/k8s$ kubectl get serviceaccount --namespace api-access
NAME          SECRETS    AGE
default       0          2m37s
pod-viewer    0          3s
```

3. Role 생성

```
# Create Role
kubectl create role podreader-role \
--resource=pod \
--verb=watch,list,get \
--namespace=api-access

# Verify Role creation
kubectl get role --namespace api-access
```

역할 기반 제어

```
cka@master:~/k8s$ kubectl get role --namespace api-access
NAME                CREATED AT
podreader-role      2026-05-11T09:31:53Z
```

4. RoleBinding 생성

```
# Create Rolebinding
kubectl create rolebinding podreader-rolebinding \
--role=podreader-role \
--serviceaccount=api-access:pod-viewer \
--namespace=api-access

# Verify Rolebinding creation
kubectl get rolebinding --namespace api-access
```

```
cka@master:~/k8s$ kubectl get rolebinding --namespace api-access
NAME                ROLE                AGE
podreader-rolebinding  Role/podreader-role  4s
```

역할 기반 제어

[ServiceAccount, ClusterRole, ClusterRoleBinding 생성]

애플리케이션 배포를 위해 새로운 **ClusterRole**을 생성하고 특정 namespace의 **ServiceAccount**를 바인딩하시오.

1. 다음의 resource type에서만 Create가 허용된 **ClusterRole** deployment-clusterrole을 생성합니다.

Resource Type: Deployment StatefulSet DaemonSet

2. 미리 생성된 namespace api-access 에 cicd-token이라는 새로운 **ServiceAccount**를 만듭니다.

3. ClusterRole deployment-clusterrole을 namespace api-access 로 제한된 새 ServiceAccountcicd-token에 **바인딩**하세요

1. NameSpace 생성

```
# Create namespace
kubectl create namespace api-access

# Verify namespace creation
kubectl get namespace
```

```
cka@master:~/k8s$ kubectl get namespace
NAME          STATUS   AGE
api-access    Active   3s
```

2. Service Account 생성

```
# Create ServiceAccount
kubectl create serviceaccount cicd-token --namespace=api-access

# Verify ServiceAccount creation
kubectl get serviceaccount --namespace=api-access
```

```
cka@master:~/k8s$ kubectl get serviceaccount --namespace=api-access
NAME          SECRETS   AGE
cicd-token    0         4s
default       0         50s
```

3. ClusterRole 생성

```
# Create ClusterRole
kubectl create clusterrole deployment-clusterrole \
--resource=deployment,statefulset,daemonset \
--verb=create

# Verify ClusterRole creation
kubectl get clusterrole | grep deployment-clusterrole
# detail: kubectl describe clusterrole deployment-clusterrole
```


역할 기반 제어

```
cka@master:~/k8s$ kubectl get clusterrole | grep deployment-clusterrole
deployment-clusterrole 2026-05-11T09:34:57Z
```

4. ClusterRoleBinding 생성

```
# Create ClusterRoleBinding
kubectl create clusterrolebinding deployment-clusterrolebinding \
--clusterrole=deployment-clusterrole \
--serviceaccount=api-access:cicd-token

# Verify ClusterRoleBinding creation
kubectl get clusterrolebinding | grep deployment-clusterrolebinding
```

```
cka@master:~/k8s$ kubectl get clusterrolebinding | grep deployment-clusterrolebinding
deployment-clusterrolebinding ClusterRole/deployment-clusterrole
23s
```

노드 관리

[노드 비우기]

k8s-worker2 노드를 스케줄링 불가능하게 설정하고,
해당 노드에서 실행 중인 모든 Pod을 다른 node로 **reschedule** 하세요.

0. 설정 전 worker-2 노드 확인

```
kubectl get node
```

```
cka@master:~$ kubectl get no
NAME          STATUS    ROLES          AGE    VERSION
master        Ready     control-plane   12d    v1.33.11
worker-1      Ready     <none>          12d    v1.33.11
worker-2      Ready     <none>          12d    v1.33.11
```

1. worker-2에서 실행 중인 모든 pod를 다른 node로 reschedule 설정

```
kubectl drain worker-2 --ignore-daemonsets
```

```
cka@master:~$ kubectl drain worker-2 --ignore-daemonsets
node/worker-2 cordoned
Warning: ignoring DaemonSet-managed Pods: calico-system/calico-node-k6nkh, calico-system/csi-node-y-zvzjq
evicting pod calico-system/calico-typha-56f4d4f775-zcfgs
pod/calico-typha-56f4d4f775-zcfgs evicted
node/worker-2 drained
```

2. worker 노드 확인

```
kubectl get node
```

```
cka@master:~$ kubectl get no
NAME          STATUS              ROLES          AGE    VERSION
master        Ready               control-plane   12d    v1.33.11
worker-1      Ready               <none>          12d    v1.33.11
worker-2      Ready,SchedulingDisabled <none>          12d    v1.33.11
```

3. worker-2 drain 설정 해제 및 노드 확인

```
kubectl uncordon worker-2

kubectl get node
```

```
cka@master:~$ kubectl uncordon worker-2
node/worker-2 uncordoned
cka@master:~$ kubectl get no
NAME          STATUS    ROLES          AGE    VERSION
master        Ready     control-plane   12d    v1.33.11
worker-1      Ready     <none>          12d    v1.33.11
worker-2      Ready     <none>          12d    v1.33.11
```

노드 관리

[Pod Scheduling]

다음의 조건으로 **Pod**를 생성하세요.

```
name: eshop-store
image: nginx
nodeSelector: disktype=ssd
```

[NodeSelector](#)
※ [pods/pod-nginx.yaml](#) 참고

1. 노드 라벨 부여(work1 => disktype=ssd)(work2 => disktype=hdd)

```
kubectl label node worker-1 disktype=ssd
kubectl label node worker-2 disktype=hdd
```

```
cka@master:~$ kubectl label node worker-1 disktype=ssd
kubectl label node worker-2 disktype=hdd
node/worker-1 labeled
node/worker-2 labeled
```

2. 노드 라벨 확인(라벨링 추가 여부)

```
kubectl get no -L disktype
```

```
cka@master:~$ kubectl get no -L disktype
NAME      STATUS    ROLES    AGE   VERSION   DISKTYPE
master    Ready     control-plane  12d   v1.33.11
worker-1  Ready     <none>      12d   v1.33.11  ssd
worker-2  Ready     <none>      12d   v1.33.11  hdd
```

3. Pod yaml 생성

```
kubectl run eshop-store \
--image=ghcr.io/mjc-t/nginx \
--dry-run=client -o yaml > eshop-store.yaml
```

```
cka@master:~$ kubectl run eshop-store --image=ghcr.io/mjc-t/nginx --dry-run=client -o yaml > ,
```

노드 관리

4. yaml 수정

```
vi eshop-store.yaml
```

```
cka@master:~$ vi eshop-store.yaml
apiVersion: v1
kind: Pod
metadata:
  name: eshop-store
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: eshop-store
  nodeSelector:
    disktype: ssd
```

5. 생성 및 확인

```
kubectl apply -f eshop-store.yaml
```

```
cka@master:~$ kubectl apply -f eshop-store.yaml
pod/eshop-store created
```

```
kubectl get po eshop-store -o wide
```

```
cka@master:~$ kubectl get po eshop-store -o wide
NAME          READY   STATUS    RESTARTS   AGE      IP             NODE          NOMINATED NODE
eshop-store   1/1     Running   0           6m36s    10.244.226.70  worker-1     <none>
```

파드 생성

[환경변수, command, args 적용]

cka-exam이라는 **namespace**를 만들고, 'cka-exam' namespace에 아래와 같은 **Pod**을 생성하시오.

```
podName: pod-01
image: busybox
env: CERT = "CKA-cert"
command: /bin/sh
args: "-c", "while true; do echo $(CERT); sleep 10;done"
```

1. NameSpace 생성

```
kubectl create namespace cka-exam
```

```
cka@master:~/k8s$ kubectl create namespace cka-exam
namespace/cka-exam created
```

2. Pod yaml 생성

```
# Create yaml File
kubectl run pod-01 \
--image=ghcr.io/mjc-t/busybox \
--namespace=cka-exam \
--dry-run=client -o yaml > pod-01.yaml

# Or
kubectl run pod-01 \
--image=ghcr.io/mjc-t/busybox \
--env=CERT="CKA-cert" \
--namespace=cka-exam \
--dry-run=client -o yaml > pod-01.yaml
```

```
cka@master:~/k8s$ kubectl run pod-01 \
> --image=ghcr.io/mjc-t/busybox \
> --env=CERT="CKA-cert" \
> --namespace=cka-exam \
> --dry-run=client -o yaml > pod-01.yaml
```

```
cka@master:~/k8s$ ls
pod-01.yaml
```

파드 생성

3. yaml 수정

```
# pod-01.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-01
  namespace: cka-exam
spec:
  containers:
    - image: ghcr.io/mjc-t/busybox
      name: pod-01
      env:
        - name: CERT
          value: "CKA-cert"
      command: ["/bin/sh"]
      args: ["-c", "while true; do echo $(CERT); sleep 10;done"]
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-01
  namespace: cka-exam
spec:
  containers:
    - image: ghcr.io/mjc-t/busybox
      name: pod-01
      env:
        - name: CERT
          value: CKA-cert
      command: ["/bin/sh"]
      args: ["-c", "while true; do echo $(CERT); sleep 10;done"]
```

4. Pod 생성 및 확인

```
# Create Pod
kubectl apply -f pod-01.yaml

# Verify Pod creation
kubectl get pods --namespace=cka-exam
```

```
cka@master:~/k8s$ kubectl apply -f pod-01.yaml
pod/pod-01 created
```

```
cka@master:~/k8s$ k get pod --namespace=cka-exam
NAME      READY   STATUS    RESTARTS   AGE
pod-01    1/1     Running   0           12m
```

파드 생성

[Static Pod 생성]

worker-1 노드에 nginx-static-pod.yaml 라는 이름의 **Static Pod**를 생성하세요.

podName: nginx-static-pod
image: nginx
port: 80

1. Pod yaml 생성

```
# Create yaml file
kubectl run nginx-static-pod \
--image=ghcr.io/mjc-t/nginx \
--port=80 \
--dry-run=client -o yaml > nginx-static-pod.yaml
```

```
cka@master:~/k8s$ kubectl run nginx-static-pod \
> --image=ghcr.io/mjc-t/nginx \
> --port=80 \
> --dry-run=client -o yaml > nginx-static-pod.yaml
cka@master:~/k8s$ ls
nginx-static-pod.yaml  pod-01.yaml
```

2. yaml 수정

```
# nginx-static-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-static-pod
spec:
  containers:
  - image: ghcr.io/mjc-t/nginx
    name: nginx-static-pod
    ports:
    - containerPort: 80
```

파드 생성

3. worker-1 node에서 yaml 작성

```
sudo -i
cd /etc/kubernetes/manifests

# Create the nginx static pod manifest
vi nginx-static-pod.yaml
```

```
root@worker1:~# cd /etc/kubernetes/manifests/
root@worker1:/etc/kubernetes/manifests# vi nginx-static-pod.yaml
root@worker1:/etc/kubernetes/manifests# ls
nginx-static-pod.yaml
```

4. master node에서 Pod 생성 확인

```
# Verify Static Pod creation
kubectl get pods -o wide
```

```
cka@master:~/k8s$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
nginx-static-pod-worker1	1/1	Running	0	59s	10.244.235.155	worker1

파드 생성

[로그 확인]

Pod "nginx-static-pod-k8s-worker1"의 log를 모니터링하고, 메시지를 포함하는 로그라인을 추출하세요.

추출된 결과는 ~/pod-log에 기록하세요

1. Pod 확인

```
# Verify Pods
kubectl get pods
```

```
cka@master:~/k8s$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-static-pod-worker1            1/1     Running   0           95s
```

2. 로그 모니터링 및 추출

```
# Monitor pod logs
kubectl logs nginx-static-pod-worker1

# Save pod logs to a file
kubectl logs nginx-static-pod-worker1 > /home/cka/pod-log

# Verify saved logs
cat /home/cka/pod-log
```

```
cka@master:~/k8s$ kubectl logs nginx-static-pod-worker1 > /home/cka/pod-log
cka@master:~/k8s$ cat /home/cka/pod-log
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/11 09:10:09 [notice] 1#1: using the "epoll" event method
2026/05/11 09:10:09 [notice] 1#1: nginx/1.29.8
2026/05/11 09:10:09 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/11 09:10:09 [notice] 1#1: OS: Linux 6.8.0-110-generic
2026/05/11 09:10:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/05/11 09:10:09 [notice] 1#1: start worker processes
2026/05/11 09:10:09 [notice] 1#1: start worker process 29
2026/05/11 09:10:09 [notice] 1#1: start worker process 30
2026/05/11 09:10:09 [notice] 1#1: start worker process 31
2026/05/11 09:10:09 [notice] 1#1: start worker process 32
```

파드 생성

[Multi Container Pod 생성]

4개의 컨테이너를 동작시키는 eshop-frontend Pod를 생성하시오.

pod image: nginx, redis, memcached, busybox

1. Pod yaml 생성

```
# Create yaml
kubectl run eshop-frontend \
--image=ghcr.io/mjc-t/nginx \
--dry-run=client -o yaml > eshop-frontend.yaml
```

```
cka@master:~/k8s$ kubectl run eshop-frontend \
> --image=ghcr.io/mjc-t/nginx \
> --dry-run=client -o yaml > eshop-frontend.yaml
cka@master:~/k8s$ ls
eshop-frontend.yaml  nginx-static-pod.yaml  pod-01.yaml
```

2. yaml 수정

```
# eshop-frontend.yaml
apiVersion: v1
kind: Pod
metadata:
  name: eshop-frontend
spec:
  containers:
    - image: ghcr.io/mjc-t/nginx
      name: nginx
    - image: ghcr.io/mjc-t/redis:3.2-alpine
      name: redis
    - image: ghcr.io/mjc-t/memcached:latest
      name: memcached
    - image: ghcr.io/mjc-t/busybox:1.28
      name: busybox
      command: ["sh", "-c", "sleep 3600"]
```

```
apiVersion: v1
kind: Pod
metadata:
  name: eshop-frontend
spec:
  containers:
    - image: ghcr.io/mjc-t/nginx
      name: nginx
    - image: ghcr.io/mjc-t/redis:3.2-alpine
      name: redis
    - image: ghcr.io/mjc-t/memcached:latest
      name: memcached
    - image: ghcr.io/mjc-t/busybox:1.28
      name: busybox
      command: ["sh", "-c", "sleep 3600"]
```

파드 생성

※ **busybox** 이미지는 실행 중인 **foreground** 프로세스가 있어야 살아있기 때문에 **command**를 달아준다.

3. 생성된 Pod 확인

```
# Verify Pod creation
kubectl get pods
```

```
cka@master:~$ kubectl get pods
NAME             READY   STATUS    RESTARTS   AGE
eshop-frontend   4/4     Running   0           31s
```

Kubernetes Controller & Networking

[Rolling Update & Roll Back]

Deployment를 이용해 nginx 파드를 3개 배포한 다음 컨테이너 이미지 버전을 **rolling update**하고 **update record**를 기록합니다.

마지막으로 컨테이너 이미지를 previous version으로 **roll back** 합니다.

name: eshop-payment

image: nginx

image version: 1.16

update image version: 1.17

label: app=payment, environment=production

1. Deployment yaml 생성

```
# Create yaml
kubectl create deploy eshop-payment \
--image=ghcr.io/mjc-t/nginx:1.16 \
--replicas=3 \
--dry-run=client -o yaml > eshop-payment.yaml
```

```
cka@master:~$ kubectl create deploy eshop-payment --image=ghcr.io/mjc-t/nginx:1.16 --replicas=3 --dry-run=client -o yaml > eshop-payment.yaml
cka@master:~$ ls
calico-bundle.tar  calico-node.tar  eshop-payment.yaml
```

2. yaml 수정

```
# eshop-payment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: payment
    environment: production
  name: eshop-payment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment
      environment: production
  template:
    metadata:
      labels:
        app: payment
        environment: production
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx:1.16
          name: nginx
```

Kubernetes Controller & Networking

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: payment
    environment: production
    name: eshop-payment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: payment
      environment: production
  template:
    metadata:
      labels:
        app: payment
        environment: production
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx:1.16
          name: nginx
```

3. Deployment 생성 및 확인

```
# Create Deployment
kubectl apply -f eshop-payment.yaml --record

# Verify Deployment, Pods creation
kubectl get deployment,pod
```

```
cka@master:~$ kubectl apply -f eshop-payment.yaml --record
Flag --record has been deprecated, --record will be removed in the future
deployment.apps/eshop-payment created
```

```
cka@master:~$ k get deploy,pod
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/eshop-payment	3/3	3	3	2m36s

NAME	READY	STATUS	RESTARTS	AGE
pod/eshop-payment-67df6b758-jnv79	1/1	Running	0	2m36s
pod/eshop-payment-67df6b758-p2st2	1/1	Running	0	2m36s
pod/eshop-payment-67df6b758-t59xz	1/1	Running	0	2m36s

4. Rolling Update 적용 및 확인

```
# Rolling Update: nginx version up
kubectl set image deployments eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record

# Verify Rolling Update
kubectl rollout history deployment eshop-payment
```

```
deployment.apps/eshop-payment image updated
cka@master:~$ kubectl rollout history deployment eshop-payment
deployment.apps/eshop-payment
REVISION  CHANGE-CAUSE
1          kubectl apply --filename=deploy.yaml --record=true
2          kubectl set image deployments eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record=true
```

Kubernetes Controller & Networking

5. Roll Back 및 확인

```
# RollBack
kubectl rollout undo deployment eshop-payment

# Verify
kubectl get pods
kubectl describe pod | grep -i image:
kubectl rollout history deployment eshop-payment
```

```
cka@master:~$ kubectl rollout undo deploy eshop-payment
deployment.apps/eshop-payment rolled back
```

```
cka@master:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
eshop-payment-544f74f7dc-2hwbp      1/1     Running   0           73s
eshop-payment-544f74f7dc-8jrn4      1/1     Running   0           72s
eshop-payment-544f74f7dc-r6cfx      1/1     Running   0           70s
cka@master:~$ kubectl describe pod | grep -i image:
    Image:          ghcr.io/mjc-t/nginx:1.16
    Image:          ghcr.io/mjc-t/nginx:1.16
    Image:          ghcr.io/mjc-t/nginx:1.16
```

```
cka@master:~$ kubectl rollout history deployment eshop-payment
deployment.apps/eshop-payment
REVISION  CHANGE-CAUSE
2          kubectl set image deployments eshop-payment nginx=ghcr.io/mjc-t/nginx:1.17 --record=true
3          kubectl apply --filename=deploy.yaml --record=true
```

Kubernetes Controller & Networking

[ClusterIP]

'devops' namespace에서 **deployment** 'eshop-order'를 다음 조건으로 생성하시오.

```
name: eshop-order
kind: deployment
namespace: devops
image: nginx
replicas: 2
label: name=order
```

eshop-order' deployment의 **Service**를 만드세요.

```
serviceName: eshop-order-svc
type: ClusterIP
port: 80
```

1. Namespace 생성

```
# Create namespace
kubectl create namespace devops

# Verify namespace creation
kubectl get namespace
```

```
cka@master:~$ kubectl create ns devops
namespace/devops created
cka@master:~$ kubectl get ns devops
NAME      STATUS   AGE
devops    Active   4s
```

2. Deployment yaml 생성

```
# Create Deployment yaml
kubectl create deployment eshop-order \
--image=ghcr.io/mjc-t/nginx \
--replicas=2 \
--namespace=devops \
--dry-run=client -o yaml > eshop-order.yaml
```

```
cka@master:~$ kubectl create deployment eshop-order \
--image=ghcr.io/mjc-t/nginx \
--replicas=2 \
--namespace=devops \
--dry-run=client -o yaml > eshop-order.yaml
cka@master:~$ ls | grep eshop-order.yaml
eshop-order.yaml
```

Kubernetes Controller & Networking

3. yaml 수정

```
# eshop-order.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    name: order
    name: eshop-order
    namespace: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      name: order
  template:
    metadata:
      labels:
        name: order
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    name: order
    name: eshop-order
    namespace: devops
spec:
  replicas: 2
  selector:
    matchLabels:
      name: order
  template:
    metadata:
      labels:
        name: order
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

4. Deployment 생성 및 확인

```
kubectl apply -f eshop-order.yaml
kubectl get deploy,pod -n devops
```

```
cka@master:~$ kubectl apply -f eshop-order.yaml
deployment.apps/eshop-order created
cka@master:~$ kubectl get deploy,pod -n devops
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/eshop-order         2/2     2            2           38s

NAME                                READY   STATUS    RESTARTS   AGE
pod/eshop-order-764ff544cd-d9r8x    1/1     Running   0          38s
pod/eshop-order-764ff544cd-k28hr    1/1     Running   0          38s
```


Kubernetes Controller & Networking

5. 서비스 생성 및 확인

```
# Create Service
kubectl expose deployment -n devops eshop-order \
--name=eshop-order-svc \
--port=80 \
--target-port=80
```

```
# Verify Service creation
kubectl get svc -n devops
```

```
cka@master:~$ kubectl expose deploy eshop-order -n devops --name=eshop-order-svc --port=80 --target-port=80
service/eshop-order-svc exposed
cka@master:~$ kubectl get svc -n devops
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
eshop-order-svc	ClusterIP	10.101.82.59	<none>	80/TCP	18s

6. 동작 확인

```
# curl [ClusterIP]
curl 10.101.82.59
```

```
cka@master:~$ curl 10.101.82.59
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Kubernetes Controller & Networking

[NodePort]

'front-end' **deployment**를 다음 조건으로 생성하시오.

image: nginx

replicas: 2

label: run=nginx

'front-end' deployment의 nginx 컨테이너를 expose하는 'front-end-nodesvc'라는 새 **service**를 만듭니다.

Front-end로 동작중인 Pod에는 node의 **30200 포트**로 접속되어야 합니다

1. Deployment yaml 생성

```
kubectl create deployment front-end \
--image ghcr.io/mjc-t/nginx \
--replicas 2 \
--dry-run=client -o yaml > front-end.yaml
```

```
cka@master:~$ kubectl create deploy front-end --image=ghcr.io/mjc-t/nginx --replicas=2 --dry-run=client -o yaml > front-end.yaml
```

2. yaml 수정

```
# front-end.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    run: nginx
  name: front-end
spec:
  replicas: 2
  selector:
    matchLabels:
      run: nginx
  template:
    metadata:
      labels:
        run: nginx
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

Kubernetes Controller & Networking

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    run: nginx
    name: front-end
spec:
  replicas: 2
  selector:
    matchLabels:
      run: nginx
  template:
    metadata:
      labels:
        run: nginx
    spec:
      containers:
        - image: ghcr.io/mjc-t/nginx
          name: nginx
```

3. 생성 및 확인

```
# Create and Verify
kubectl apply -f front-end.yaml
kubectl get deploy,pod
```

```
cka@master:~$ kubectl apply -f front-end.yaml
deployment.apps/front-end created
cka@master:~$ kubectl get deploy,pod
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/eshop-payment	3/3	3	3	25m
deployment.apps/front-end	2/2	2	2	34s

NAME	READY	STATUS	RESTARTS	AGE
pod/eshop-payment-67df6b758-dtd52	1/1	Running	0	18m
pod/eshop-payment-67df6b758-kdnmd	1/1	Running	0	18m
pod/eshop-payment-67df6b758-tb5g4	1/1	Running	0	18m
pod/front-end-7d7b4bb7bb-4567x	1/1	Running	0	34s
pod/front-end-7d7b4bb7bb-9q9h6	1/1	Running	0	34s

Kubernetes Controller & Networking

4. Service yaml 생성

```
# Create Service yaml
k expose deployment front-end --name front-end-nodesvc \
--port 80 \
--target-port 80 \
--type NodePort \
--dry-run=client -o yaml > front-end-nodesvc.yaml
```

```
cka@master:~$ kubectl expose deploy front-end --name=front-end-nodesvc --port=80 --target-port=80 -
-type=NodePort --dry-run=client -o yaml > front-end-nodesvc.yaml
```

5. yaml 수정

```
# front-end-nodesvc.yaml
apiVersion: v1
kind: Service
metadata:
  labels:
    run: nginx
  name: front-end-nodesvc
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
      nodePort: 30200
  selector:
    run: nginx
  type: NodePort
```

```
apiVersion: v1
kind: Service
metadata:
  labels:
    run: nginx
  name: front-end-nodesvc
spec:
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
      nodePort: 30200
  selector:
    run: nginx
  type: NodePort
```

Kubernetes Controller & Networking

6. Service 생성 및 확인

```
# Create and Verify
kubectl apply -f front-end-nodesvc.yaml
kubectl get svc
```

```
cka@master:~$ kubectl apply -f front-end-nodesvc.yaml
service/front-end-nodesvc created
cka@master:~$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
front-end-nodesvc	NodePort	10.100.62.38	<none>	80:30200/TCP	14s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	55m

7. NodePort 동작 확인

```
# curl [NodeIP]:[NodePort]
curl 192.168.111.20:30200
```

```
cka@master:~$ curl 192.168.111.20:30200
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

Kubernetes Controller & Networking

[Pod Scale-out]

실행되고 있는 eshop-order 디플로이먼트의 파드 숫자를 **5개**로 늘리시오.

namespace: devops

deployment: eshop-order - replicas: 5

1. Deployment 확인

```
# Get
kubectl get deployment -n devops
```

```
cka@master:~$ kubectl get deploy -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
eshop-order    2/2     2            2           18m
```

2. Deployment 파드 늘리기

```
# Scale deployment to 5 replicas
kubectl scale deploy eshop-order -n devops --replicas=5
```

```
cka@master:~$ kubectl scale deploy eshop-order -n devops --replicas=5
deployment.apps/eshop-order scaled
```

3. 결과 확인

```
# Verify
kubectl get deploy,pod -n devops
```

```
cka@master:~$ kubectl get deploy,pod -n devops
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/eshop-order    5/5     5            5           19m

NAME          READY   STATUS    RESTARTS   AGE
pod/eshop-order-764ff544cd-6txlw  1/1     Running   0          24s
pod/eshop-order-764ff544cd-cf2xb  1/1     Running   0          24s
pod/eshop-order-764ff544cd-d9r8x  1/1     Running   0          19m
pod/eshop-order-764ff544cd-dzxtv  1/1     Running   0          24s
pod/eshop-order-764ff544cd-k28hr  1/1     Running   0          19m
```

NetworkPolicy & Ingress

[Network Policy]

customera, customerb를 생성한 후, 각각 PARTITION=customera, PARTITION=customerb를 라벨링하시오.

default namespace에 다음과 같은 **Pod**를 생성하세요.

name: poc
image: nginx
port: 80
label: app=po

"partition=customera"를 사용하는 namespace에서만 poc의 80포트로 연결할 수 있도록 default namespace에 'allow-web-from-customera'라는 **network Policy**를 설정하세요.

보안 정책상 다른 namespace의 접근은 제한합니다.

[Network Policies](#)

※ [service/networking/networkpolicy.yaml](#) 참고

1. NameSpace 생성 및 확인

```
kubectl create ns customera
kubectl create ns customerb

kubectl get ns | grep customer
```

```
cka@master:~$ k get ns | grep customer
customera      Active    7s
customerb      Active    7s
```

2. 라벨링 및 확인

```
kubectl label ns customera partition=customera
kubectl label ns customerb partition=customerb

kubectl get ns --show-labels | grep customer
```

```
cka@master:~$ kubectl get ns --show-labels | grep customer
customera      Active    2m45s   kubernetes.io/metadata.name=customera,partition=customera
customerb      Active    2m45s   kubernetes.io/metadata.name=customerb,partition=customerb
```

NetworkPolicy & Ingress

3. yaml 작성

```
# netpol.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-from-customer
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: poc
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              partition: customer
      ports:
        - protocol: TCP
          port: 80
```

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-web-from-customer
  namespace: default
spec:
  podSelector:
    matchLabels:
      app: poc
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              partition: customer
      ports:
        - protocol: TCP
          port: 80
```

4. 생성 및 확인

```
kubectl apply -f netpol.yaml

kubectl get networkpolicy
```

```
cka@master:~$ kubectl apply -f netpol.yaml
networkpolicy.networking.k8s.io/allow-web-from-customer unchanged
cka@master:~$ kubectl get netpol
NAME                                POD-SELECTOR  AGE
allow-web-from-customer             app=poc        3d
```


NetworkPolicy & Ingress

[Ingress]

Create a new nginx **Ingress** resource as follows:

name: ping
namespace: ing-internal - Exposing service hi on path /hi using service port 5678

[Ingress](#)

※ [service/networking/minimal-ingress.yaml](#) 참고

1. NameSpace 생성 및 확인

```
kubectl create namespace ing-internal  
  
kubectl get namespace
```

```
cka@master:~$ kubectl create ns ing-internal  
namespace/ing-internal created  
cka@master:~$ kubectl get ns ing-internal  
NAME          STATUS    AGE  
ing-internal   Active    7s
```

2. yaml 작성

```
# ing.yaml  
apiVersion: networking.k8s.io/v1  
kind: Ingress  
metadata:  
  name: ping  
  namespace: ing-internal  
spec:  
  ingressClassName: nginx-example  
  rules:  
    - http:  
        paths:  
          - path: /hi  
            pathType: Prefix  
            backend:  
              service:  
                name: hi  
                port:  
                  number: 5678
```

NetworkPolicy & Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ping
  namespace: ing-internal
spec:
  ingressClassName: nginx-example
  rules:
    - http:
        paths:
          - path: /hi
            pathType: Prefix
            backend:
              service:
                name: hi
                port:
                  number: 5678
```

3. 생성 및 확인

```
kubectl apply -f ing.yaml

kubectl get ingress -n ing-internal
```

```
cka@master:~$ kubectl apply -f ing.yaml
ingress.networking.k8s.io/ping created
cka@master:~$ kubectl get ing -n ing-internal
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ping	nginx-example	*		80	36s

NetworkPolicy & Ingress

[Service and DNS Lookup]

image nginx를 사용하는 resolver pod를 생성하고 resolver-service라는 service를 구성합니다.
클러스터 내에서 service와 pod 이름을 조회할 수 있는지 테스트합니다.

dns 조회에 사용하는 pod 이미지는 **busybox:1.28**
service와 pod 이름 조회는 **nslookup**을 사용합니다.
service 조회 결과는 **~/nginx.svc**에 pod name 조회 결과는 **~/nginx.pod** 파일에 기록합니다.

1. Pod 생성

```
kubectl run resolver --image ghcr.io/mjc-t/nginx --port 80  
  
kubectl get pods resolver
```

```
cka@master:~$ kubectl run resolver --image=ghcr.io/mjc-t/nginx --port=80  
pod/resolver created  
cka@master:~$ kubectl get pod resolver  
NAME          READY   STATUS    RESTARTS   AGE  
resolver      1/1     Running   0           97s
```

2. Service 생성

```
kubectl expose pod resolver --name resolver-service --port 80  
  
kubectl get svc resolver-service
```

```
cka@master:~$ kubectl expose pod resolver --name=resolver-service --port=80  
service/resolver-service exposed  
cka@master:~$ kubectl get svc resolver-service  
NAME             TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE  
resolver-service ClusterIP     10.108.94.23  <none>       80/TCP     78s
```

3. DNS Lookup - Service

```
# service  
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox \  
# --restart=Never --rm \  
# -- nslookup [ClusterIP]  
  
kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox \  
--restart=Never --rm \  
-- nslookup 10.108.94.23 > ~/nginx.svc
```

NetworkPolicy & Ingress

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox:1.28 --restart=Never --rm -- nslookup
10.108.94.23
If you don't see a command prompt, try pressing enter.
warning: couldn't attach to pod/test-nslookup, falling back to streaming logs: Internal error occurred: unable
to upgrade connection: container test-nslookup not found in pod test-nslookup_default
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: 10.108.94.23
Address 1: 10.108.94.23 resolver-service.default.svc.cluster.local
pod "test-nslookup" deleted
```

3. DNS Lookup - Pod

```
# pod
# kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox \
# --restart=Never --rm \
# -- nslookup [PodIP].[namespace].pod.cluster.local

kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox \
--restart=Never --rm \
-- nslookup 10-244-235-191.default.pod.cluster.local > ~/nginx.pod
```

```
cka@master:~$ kubectl get po resolver -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE      NOMINATED NODE   READINESS GATES
resolver  1/1     Running   0           6m7s  10.244.235.191 worker1      <none>           <none>

cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox:1.28 --restart=Never --rm -- nslookup
10-244-235-191.default.pod.cluster.local
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: 10-244-235-191.default.pod.cluster.local
Address 1: 10.244.235.191
pod "test-nslookup" deleted
```

4. 파일 확인

[Service]

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox:1.28 --restart=Never --rm -- nslookup
10.108.94.23 > nginx.svc
cka@master:~$ ls
calico-bundle.tar calico-node.tar nginx.svc
cka@master:~$ cat nginx.svc
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: 10.108.94.23
Address 1: 10.108.94.23 resolver-service.default.svc.cluster.local
pod "test-nslookup" deleted
```

[Pod]

```
cka@master:~$ kubectl run -it test-nslookup --image=ghcr.io/mjc-t/busybox:1.28 --restart=Never --rm -- nslookup
10-244-235-191.default.pod.cluster.local > nginx.pod
cka@master:~$ ls
calico-bundle.tar calico-node.tar nginx.pod nginx.svc
cka@master:~$ cat nginx.pod
Server: 10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name: 10-244-235-191.default.pod.cluster.local
Address 1: 10.244.235.191
pod "test-nslookup" deleted
```

Resource 관리

[emptyDir Volume]

다음 조건에 맞춰서 nginx 웹서버 **Pod**가 생성한 로그파일을 받아서 STDOUT으로 출력하는 busybox 컨테이너를 운영하시오.

emptyDir 볼륨을 통한 데이터 공유

podName: weblog

Web container:

image: nginx:1.17

volumeMount: /var/log/nginx

readwrite

Log container:

image: busybox

args: /bin/sh, -c, "tail -n+1 -f /data/access.log" - Volume mount : /data

readonly

1. Pod yaml 생성

```
kubect1 run weblog \  
--image ghcr.io/mjc-t/nginx:1.17 \  
--dry-run=client -o yaml > weblog.yaml
```

```
cka@master:~$ kubect1 run weblog \  
> --image ghcr.io/mjc-t/nginx:1.17 \  
> --dry-run=client -o yaml > weblog.yaml  
cka@master:~$ ls | grep weblog  
weblog.yaml
```

Resource 관리

2. yaml 수정

```
# weblog.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: weblog
  name: weblog
spec:
  containers:
    - image: ghcr.io/mjc-t/nginx:1.17
      name: web
      volumeMounts:
        - mountPath: /var/log/nginx
          name: weblog
    - image: ghcr.io/mjc-t/busybox
      name: log
      args: ["/bin/sh", "-c", "tail -n+1 -f /data/access.log"]
      volumeMounts:
        - mountPath: /data
          name: weblog
          readOnly: true
  volumes:
    - name: weblog
      emptyDir: {}
```

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: weblog
  name: weblog
spec:
  containers:
    - image: ghcr.io/mjc-t/nginx:1.17
      name: web
      volumeMounts:
        - mountPath: /var/log/nginx
          name: weblog
    - image: ghcr.io/mjc-t/busybox
      name: log
      args: ["/bin/sh", "-c", "tail -n+1 -f /data/access.log"]
      volumeMounts:
        - mountPath: /data
          name: weblog
          readOnly: true
  volumes:
    - name: weblog
      emptyDir: {}
```

3. 생성 및 확인

```
kubectl apply -f weblog.yaml
```

```
kubectl get pods
```

```
cka@master:~$ kubectl get pods weblog
NAME      READY   STATUS    RESTARTS   AGE
weblog    2/2     Running   0           42s
```

Resource 관리

[HostPath Volume]

~/fluentd.yaml 파일을 만들어 새로운 **Pod** 생성하세요

podName: fluentd

image: fluentd

namespace: default

위 조건을 참고하여 다음 조건에 맞게 **볼륨마운트**를 설정하세요.

1. Worker node의 도커 컨테이너 디렉토리 : /var/lib/docker/containers 동일 디렉토리로 pod에 마운트 하세요.
2. Worker node의 /var/log 디렉토리를 fluentd Pod에 동일이름의 디렉토리 마운트하세요.

1. Pod yaml 생성

```
kubectl run fluentd \
--image ghcr.io/mjc-t/fluentd \
--dry-run=client -o yaml > fluentd.yaml
```

```
cka@master:~/k8s$ k run fluentd \
> --image=ghcr.io/mjc-t/fluentd \
> --dry-run=client -o yaml > fluentd.yaml
cka@master:~/k8s$ ls
eshop-frontend.yaml  fluentd.yaml  nginx-static-pod.yaml  pod-01.yaml  weblog.yaml
```

2. yaml 수정

```
# fluentd.yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: fluentd
  name: fluentd
spec:
  containers:
  - image: ghcr.io/mjc-t/fluentd
    name: fluentd
    ports:
    - containerPort: 80
  volumeMounts:
  - name: containersdir
    mountPath: /var/lib/docker/containers
  - name: logdir
    mountPath: /var/log
  volumes:
  - name: containersdir
    hostPath:
      path: /var/lib/docker/containers
  - name: logdir
    hostPath:
      path: /var/log
```

Resource 관리

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: fluentd
    name: fluentd
spec:
  containers:
    - image: ghcr.io/mjc-t/fluentd
      name: fluentd
      ports:
        - containerPort: 80
      volumeMounts:
        - name: containersdir
          mountPath: /var/lib/docker/containers
        - name: logdir
          mountPath: /var/log
  volumes:
    - name: containersdir
      hostPath:
        path: /var/lib/docker/containers
    - name: logdir
      hostPath:
        path: /var/log
```

3. 생성 및 확인

```
kubectl apply -f fluentd.yaml

kubectl get pods
```

```
cka@master:~$ kubectl get pods fluentd
NAME      READY   STATUS    RESTARTS   AGE
fluentd   1/1     Running   0           68s
```


Resource 관리

[Persistent Volume]

pv001라는 이름으로 size **1Gi**, access mode **ReadWriteMany**를 사용하여 **persistent volume**을 생성합니다.

volume type은 **hostPath**이고 위치는 **/tmp/app-config**입니다.

[PersistentVolume](#)

※ [pods/storage/pv-volume.yaml](#) 참고

1. Persistent Volume yaml 작성

```
# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/tmp/app-config"
```

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: "/tmp/app-config"
```

2. 생성 및 확인

```
kubectl apply -f pv.yaml

kubectl get pv
```

```
cka@master:~$ kubectl apply -f pv.yaml
persistentvolume/pv001 created
cka@master:~$ kubectl get pv
NAME      CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS   CLAIM   STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
pv001    1Gi       RWX           Retain          Available             storageclass  <unset>              5s
```

Resource 관리

[PVC를 사용하는 애플리케이션 Pod 운영]

다음의 조건에 맞는 새로운 **PersistentVolumeClaim** 생성하시오.

name: pv-volume
class: app-hostpath-sc
capacity: 10Mi

앞서 생성한 pv-volume PersistentVolumeClaim을 mount하는 **Pod**를 생성하시오.

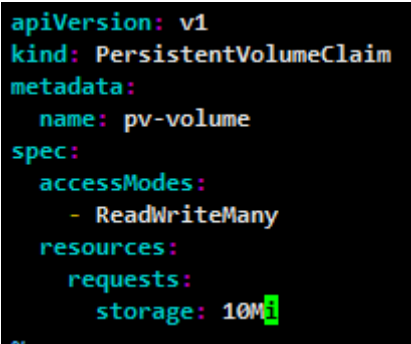
name: web-server-pod
image: nginx
mountPath: /usr/share/nginx/html
volume에서 **ReadWriteMany** 액세스 권한을 가지도록 구성합니다

[PersistentVolumeClaim](#)

※ **PersistentVolumeClaims, Claims As Volumes** 참고

1. PVC yaml 작성

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pv-volume
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Mi
```



Resource 관리

2. 생성 및 확인

```
kubectl apply -f pvc.yaml

kubectl get pvc
```

```
cka@master:~$ kubectl apply -f pvc.yaml
persistentvolumeclaim/pv-volume created
cka@master:~$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
pv-volume	Bound	pv001	1Gi	RWX		<unset>	3s

3. Pod yaml 작성

```
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-server-pod
spec:
  containers:
    - name: nginx
      image: ghcr.io/mjc-t/nginx
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pv-volume
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server-pod
spec:
  containers:
    - name: nginx
      image: ghcr.io/mjc-t/nginx
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pv-volume
```

4. 생성 및 확인

```
kubectl apply -f pod.yaml

kubectl get pods
```

```
cka@master:~$ kubectl apply -f pod.yaml
pod/web-server-pod created
cka@master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-server-pod	1/1	Running	0	5s



HISTORY
